Day 6–7: Model Validation, Overfitting & Model Selection

Concise Lecture Notes

Slide 1 — Title: Day 6–7: Model Validation, Overfitting & Model Selection

Topic: Building models that work beyond the notebook

Key Focus:

- Understanding why models fail in production
- Detecting and preventing overfitting and data leakage
- Selecting models intelligently instead of randomly
- Explaining decisions clearly in interviews

Slide 2 — Why These Two Days Matter

What You've Learned So Far:

- ML fundamentals
- EDA
- Training models
- Evaluating with metrics

Critical Question Now: "Will this model work on new, real-world data?"

Why This Matters:

- Models can show 99% accuracy in training but fail in production
- Proper validation prevents costly deployment failures
- Determines if you're doing ML engineering vs just experimenting

---

Slide 3 — The Core Problem

The Paradox: A model can perform extremely well on training data but fail completely in production.

Three Main Culprits:

1. **Overfitting:** Model memorizes training data instead of learning patterns
2. **Poor Validation:** Testing on same data used for training
3. **Data Leakage:** Using information that won't be available at prediction time

Key Insight: These are engineering problems, not math problems.

PART 1 — TRAIN / VALIDATION SPLIT (DAY 6)

Slide 4 — Why We Split Data

Three Reasons:

1. **Measure Generalization:** How well does the model work on new data?
2. **Detect Overfitting Early:** Catch problems before production

3. **Simulate Real-World Performance:** Validation mimics production conditions

Golden Rule: If you train and test on the same data, the metric is meaningless.

Example:

> Without split: Train on 1000 houses, test on same 1000 → 99% accuracy (meaningless)
> With split: Train on 700, test on 300 → 83% accuracy (honest estimate)

Slide 5 — Common Data Splits

Two-Way Split:

- Training: 70-80%
- Test: 20-30%
- Use for simple projects

Three-Way Split (Recommended):

- Training: 70%
- Validation: 15%
- Test: 15%

Purpose of Each:

- **Training:** Model learns from this
- **Validation:** Guide decisions, tune hyperparameters (use multiple times)
- **Test:** Final evaluation only (use ONCE)

Critical Rules:

- Never train on validation or test data
- Never make decisions based on test data
- Look at test data only once at the very end

Slide 6 — Simple Example (Regression)

Problem: Predicting house prices

Wrong Approach:

```
python

# Train on ALL data
model.fit(all_data)
# Test on SAME data
rmse = evaluate(all_data)  # $12,500 - looks great!
# Deploy to production → Fails with $187,000 error!
```

Correct Approach:

```
python
```

```
# Split first
train, val = split(data)
# Train on subset
model.fit(train)
# Test on unseen data
rmse = evaluate(val)  # $18,700 - honest estimate
# Production → ~$20,000 error (as expected)
```

Key Insight: The "worse" validation performance is actually more honest and useful.

Slide 7 — Hands-On Exercise (Splitting Data)

Exercise: Use diamond dataset

Steps:

1. Split into train (70%) and validation (30%)
2. Train model on training set only
3. Evaluate on both sets
4. Compare metrics

Sample Results:

```
Training RMSE: $1,235
Validation RMSE: $1,289
Gap: 4.4% → Good generalization!

vs.

Training RMSE: $856
Validation RMSE: $2,146
Gap: 150% → Severe overfitting!
```

Interpretation Guidelines:

- Gap < 10%: Good
- Gap 10-20%: Acceptable
- Gap 20-50%: Warning
- Gap > 50%: Severe overfitting

Slide 8 — What Is Overfitting?

Definition: Model learns noise instead of signal (memorization vs learning)

Analogy: Student memorizes practice test answers (100% on practice) but fails real exam (45%) because questions are different.

Three Characteristics:

1. Training error is very low
2. Validation error is high
3. Large gap between them

Example:

```
Overfitted Model:
- Training RMSE: $500 (amazing!)
- Validation RMSE: $15,000 (terrible!)
- Learned specific examples, not general patterns
```

Slide 9 — Underfitting vs Overfitting

Underfitting (Too Simple):

- Training error: High
- Validation error: High
- Gap: Small
- Problem: Model can't capture patterns
- Solution: Add complexity

Well-Fitted (Just Right):

- Training error: Moderate
- Validation error: Similar to training
- Gap: Small
- This is the goal!

Overfitting (Too Complex):

- Training error: Very low
- Validation error: High
- Gap: Large
- Problem: Model memorized training data
- Solution: Reduce complexity

Visual Analogy:

- Underfitted: Straight line trying to fit a curve
- Well-fitted: Smooth curve following the general trend
- Overfitted: Wiggly line through every single point

Slide 10 — Signs of Overfitting

Key Warning Signs:

1. **Training accuracy >> Validation accuracy**
   - Gap > 20% is definite overfitting
2. **Training error << Validation error**
   - Validation 2x+ worse than training
3. **Model performance degrades in production**
   - Development: 94% accuracy
   - Production: 65% accuracy
4. **Suspiciously good training metrics**
   - 99%+ accuracy on complex real-world problem

Detection Methods:

- Monitor train vs validation gap
- Check learning curves
- Use cross-validation
- Review feature importance

Slide 11 — How Overfitting Happens

Four Main Causes:

1. Too Many Features

- 200 features with 500 samples → Model finds spurious correlations
- Rule of thumb: Need 10+ samples per feature

2. Too Complex Models

- Deep neural network for simple linear relationship
- Decision tree with no depth limit

3. Small Datasets

- 200 samples trying to train a model with 10,000 parameters
- Not enough data to learn generalizable patterns

4. Data Leakage

- Using information that won't exist at prediction time
- Most dangerous because it's hard to detect

Key Point: Overfitting is often accidental, not intentional.

Slide 12 — Preventing Overfitting

Five Main Techniques:

1. Simpler Models

Start simple, add complexity only if needed:
Linear Regression → Ridge/Lasso → Decision Tree → Random Forest

2. Regularization

```python
# Ridge (L2): Shrinks all coefficients
Ridge(alpha=1.0)

# Lasso (L1): Forces some coefficients to zero, performs feature selection
Lasso(alpha=1.0)
```

3. Feature Selection

```python
# Keep only important features
SelectKBest(k=20)  # Keep top 20 features
```

4. More Data

- More samples dilute the noise
- 500 samples → 99% train, 68% val (overfitting)
- 50,000 samples → 92% train, 91% val (good!)

5. Cross-Validation

```python
# Test on multiple splits, average results
cross_val_score(model, X, y, cv=5)
```

Slide 13 — What Is Data Leakage?

Definition: Information from outside the training process is used to build the model, artificially inflating performance.

Why Dangerous:

- Creates illusion of perfect model
- Only discovered in production (too late!)
- Wastes enormous resources
- Hard to detect without careful review

Simple Analogy: Taking an exam with the answer key attached. You score 100% but didn't learn anything. On a different exam, you fail.

Key Question for Every Feature:

> "Would this feature be available at the time I need to make a prediction in production?"

If answer is "No" → It's leakage.

Slide 14 — Common Data Leakage Examples

Example 1: Using Future Data

```python
# Predicting stock price at market open
features = [
    'yesterday_close',  # ✓ Available
    'today_close',      # ✗ FUTURE DATA!
]
```

Example 2: Preprocessing Before Splitting

```python
# WRONG
scaler.fit(all_data)  # Leaks validation stats
train, val = split(all_data)

# CORRECT
train, val = split(all_data)
scaler.fit(train)  # Only learns from training
```

Example 3: Feature Includes Target

```python
# Predicting loan default
features = [
    'income',           # ✓ Available
    'days_past_due',    # ✗ Only known AFTER default
]
```

Example 4: Duplicate Data

```python
```

```
# Same records in train and validation
# Model "predicts" by recognizing duplicates
# Solution: Remove duplicates before splitting
```

Slide 15 — Leakage Example (Realistic)

Scenario: Loan default prediction

The Problem:

```
python

features = ['income', 'credit_score', 'days_past_due']
# 'days_past_due' only exists AFTER loan is issued
```

Why It's Leakage:

- At application time: days_past_due = 0 (no payment history yet)
- In historical data: defaulted loans have high days_past_due
- Model learns: if days_past_due > 0 → predict default
- Training: 99% accuracy (perfect correlation!)
- Production: Feature doesn't exist → model fails

Timeline:

```
Day 1: Loan application (need prediction) ← days_past_due unknown
Day 30-360: Payment history develops
Day 360: Loan defaults ← days_past_due becomes available

Using data from Day 360 to predict Day 1 = time travel!
```

Impact:

- Development: 98.8% accuracy
- Production: Unusable
- Cost: $1M+ wasted

Slide 16 — Hands-On Leakage Detection Exercise

Exercise: Review diamond dataset features

For Each Feature Ask:

1. Available at prediction time?
2. Does it encode the target?
3. Is it a consequence of what we're predicting?

Diamond Features Analysis:

- `carat`, `cut`, `color`, `clarity`: ✓ All physical characteristics, available before pricing
- `depth`, `table`, `x`, `y`, `z`: ✓ Physical measurements

Hypothetical Problem Features:

- `previous_sale_price`: ⚠️ Encodes target (price predicting price)
- `insurance_value`: ✗ Calculated from market price (leakage)
- `buyer_offered_price`: ✗ Future data (comes after listing)

- `sold_date`: ✗ Only exists after sale

Detection Checklist:

- [ ] All features available at prediction time?
- [ ] No price-derived features?
- [ ] No future data?
- [ ] Preprocessing done after split?
- [ ] No duplicates in train/val?

PART 2 — MODEL SELECTION & BASELINES (DAY 7)

Slide 17 — Why Model Selection Matters

The Problem: Trying many models blindly

```python
# Bad approach
models = [LogisticRegression(), RandomForest(), GradientBoosting(),
          SVC(), KNN(), NeuralNetwork()]
# Try all, pick highest score
```

Why This Is Bad:

1. **Wastes Time:** 9+ hours trying all models vs 30 min with reasoning
2. **Increases Overfitting Risk:** One model might look good by chance
3. **Creates False Confidence:** "Best of 10" might just be lucky

Better Approach: Reasoning-based selection

The Right Process:

1. Understand the problem
2. Choose model family based on requirements
3. Start simple
4. Add complexity only if needed

Example Decision:

```
Problem: Real-time fraud detection
- Need: Fast inference, interpretable, handles imbalance
- Choose: Logistic Regression first
- Then: Random Forest if needed
- NOT: Deep neural network (too slow, not interpretable)
```

Slide 18 — Always Start with a Baseline

What is a Baseline? The simplest possible approach to your problem.

Why Baselines Matter:

1. **Provide Context**

```
Without baseline: "78% accuracy" → Is this good? 🤷
With baseline: "73% baseline, 78% model" → 5% improvement ✓
```

2. **Prevent Wasted Effort**

```
Baseline (1 day): 80% accuracy
Complex model (5 weeks): 81% accuracy
Decision: Use baseline! 1% not worth 5 weeks
```

3. **Catch Problems**

```
Baseline RMSE: $45,000
Your model RMSE: $2,500
→ Too good! Check for data leakage
```

Common Baselines:

Regression:

- Mean prediction
- Median prediction
- Simple linear regression (one feature)

Classification:

- Most frequent class
- Logistic regression

Slide 19 — Baseline Exercise

Exercise: Diamond price prediction

Step 1: Create Baseline

```python
# Always predict the mean price
mean_price = y_train.mean()  # $3,933
baseline_rmse = $3,989
```

Step 2: Train Your Model

```python
model = LinearRegression()
model.fit(X_train, y_train)
model_rmse = $1,130
```

Step 3: Compare

```
Baseline RMSE: $3,989
Model RMSE: $1,130
Improvement: 71.7%

Conclusion: Model provides significant value!
```

Decision Guidelines:

- Improvement > 20%: Model is valuable
- Improvement 5-20%: Consider cost vs benefit
- Improvement < 5%: Stick with baseline

Key Question: "Is my model actually better than doing something simple?"

Slide 20 — Model Selection Strategy

Systematic Approach:

Step 1: Define Problem Type

- Classification or regression?
- Binary or multi-class?
- Time series or static?

Step 2: Establish Baseline

- Mean/median for regression
- Most frequent class for classification
- Document baseline performance

Step 3: Try Simple Models

- Linear/Logistic regression first
- Fast, interpretable, good starting point

Step 4: Increase Complexity Only If Justified

- Simple model not good enough?
- Try Ridge/Lasso
- Then tree-based models
- Finally neural networks (if really needed)

Decision Framework:

Need interpretability? → Linear models
Small data? → Simple models (Linear, Decision Tree)
Medium data? → Tree ensembles (Random Forest)
Large data + complex patterns? → Neural networks
Text data? → NLP models
Time series? → ARIMA, LSTM

Slide 21 — Common Model Choices (Regression)

Typical Progression:

Level 1: Linear Regression

- Simplest, fastest
- Highly interpretable
- Good first choice

Level 2: Ridge / Lasso

- Adds regularization
- Prevents overfitting
- Still interpretable

Level 3: Tree-Based Models

- Random Forest, Gradient Boosting
- Handles non-linearity
- Less interpretable

Level 4: Ensemble Methods

- XGBoost, LightGBM
- High performance
- More complex

Trade-off: Each step trades interpretability for power

Slide 22 — Bias–Variance Tradeoff (Intuition)

Bias: Error from overly simple assumptions

- High bias = underfitting
- Model misses relevant patterns
- Example: Linear model for non-linear data

Variance: Error from sensitivity to data fluctuations

- High variance = overfitting
- Model captures noise
- Example: Deep decision tree memorizing training data

The Balance:

```
High Bias, Low Variance:
- Simple model
- Consistent but wrong
- Underfitting

Low Bias, High Variance:
- Complex model
- Accurate on training, fails on new data
- Overfitting

Sweet Spot:
- Moderate bias and variance
- Good generalization
```

Goal: Find the model complexity that minimizes total error (bias + variance).

Slide 23 — Cross-Validation (Conceptual)

Problem with Single Split: Might get lucky/unlucky with that one split.

Solution: Cross-Validation Split data multiple times, average performance.

5-Fold Cross-Validation:

```
Fold 1: [Val][Train][Train][Train][Train]
Fold 2: [Train][Val][Train][Train][Train]
Fold 3: [Train][Train][Val][Train][Train]
Fold 4: [Train][Train][Train][Val][Train]
Fold 5: [Train][Train][Train][Train][Val]

Average results across all folds
```

Benefits:

- More reliable performance estimate
- Uses all data for both training and validation

- Detects model instability

Example:

```
python

scores = cross_val_score(model, X, y, cv=5)
# Scores: [0.84, 0.87, 0.82, 0.86, 0.85]
# Mean: 0.848, Std: 0.018
# Consistent across folds → Good!
```

Slide 24 — Exercise: Model Comparison

Exercise: Compare two models fairly

Steps:

1. Train Model A
2. Train Model B
3. Evaluate both on same validation set
4. Compare metrics
5. Decide which to keep and explain why

Example Comparison:

```
Model A (Linear Regression):
- Val RMSE: $1,289
- Training time: 2 seconds
- Interpretable: Yes
- Feature importance: Clear

Model B (Random Forest):
- Val RMSE: $1,156
- Training time: 5 minutes
- Interpretable: Limited
- Feature importance: Available

Decision:
- Improvement: 10.3%
- Is it worth complexity? Consider:
  * Business needs
  * Deployment constraints
  * Maintenance requirements
```

Key Point: Explanation matters more than raw score. Must justify choice.

Slide 25 — Common Mistakes in Model Selection

Mistake 1: Selecting Model Based on Test Data

```
Bad: Try 10 models, pick best on test set
Problem: Test set is now contaminated
Solution: Use validation for selection, test only for final check
```

Mistake 2: Comparing Models with Different Metrics

```
Bad: "Model A has 85% accuracy, Model B has 0.78 F1"
Problem: Can't compare different metrics
Solution: Evaluate all models on same metric
```

Mistake 3: Ignoring Baseline

Bad: "My model has 78% accuracy!" (no context)
Good: "78% vs 73% baseline = 5% improvement"

Mistake 4: Over-Tuning

Bad: Tune 50 hyperparameters until validation perfect
Problem: Overfitting to validation set
Solution: Reasonable tuning, use cross-validation

Slide 26 — Interview Perspective

Common Interview Questions:

Q: "Why did you choose this model?"

Bad: "I tried many models, this worked best"
Good: "Given imbalanced data and need for interpretability,
I chose Random Forest because it handles class imbalance well
and provides feature importance for stakeholders"

Q: "What was your baseline?"

Bad: "Baseline? I just built the model"
Good: "I started with logistic regression baseline (79% accuracy).
My Random Forest achieved 87%, a meaningful 8% improvement"

Q: "How did you prevent overfitting?"

Bad: "I didn't worry about that"
Good: "I used train/val split, monitored the gap between them,
applied regularization, and used cross-validation for stability"

Q: "How did you detect leakage?"

Good: "I reviewed every feature asking 'Is this available at
prediction time?' and removed days_past_due because it only
exists after the target event"

Slide 27 — Key Takeaways

Critical Principles:

1. **Validation simulates real-world performance**
   - Always split data before training
   - Monitor train vs validation gap
   - Use cross-validation for robustness
2. **Overfitting is common and dangerous**
   - Happens when model memorizes instead of learns
   - Detected by large train/val gap
   - Prevented by simpler models, regularization, more data
3. **Data leakage ruins models silently**
   - Using future information or target-derived features
   - Creates perfect-looking but useless models
   - Check every feature: "Available at prediction time?"

4. **Baselines anchor good model selection**
    - Start with simplest approach
    - Your model must beat it
    - Provides context for performance

The Professional Approach:

- Start simple, add complexity only when justified
- Always establish baseline first
- Monitor for overfitting continuously
- Question every feature for potential leakage
- Can explain and justify every decision

Slide 28 — Homework / Practice

Before Next Session:

1. Add Train/Validation Split to Your Notebook

```python
# Proper 3-way split
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5)
```

2. Compare At Least Two Models

```python
# Baseline
baseline = LinearRegression()
# Your model
model = RandomForestRegressor()
# Compare on same validation set
```

3. Document:

- **Overfitting observations:** Train vs val gap, what it means
- **Baseline comparison:** How much better is your model?
- **Feature check:** Any potential leakage?
- **Model justification:** Why did you choose this model?

4. Practice Explaining: Be ready to answer:

- "Why this model over alternatives?"
- "How do you know it's not overfitted?"
- "What was your baseline and how much did you improve?"
- "How did you check for data leakage?"

Deliverables:

- Updated notebook with proper splits
- Comparison table: baseline vs your model
- Written justification of model choice
- Leakage check documentation

END OF DAY 6-7 LECTURE NOTES

Summary: You now understand how to build models that work beyond your notebook by:

- Properly validating with train/val/test splits
- Detecting and preventing overfitting
- Identifying and eliminating data leakage
- Selecting models systematically with baselines

Next Steps: Apply these principles to your projects and be ready to explain your validation strategy clearly!